

THREE COLORS OF MADNESS

Technical Design Document

Internal working document. Not for external distribution. Companion to the Design Bible.

Dejunai

Contents

Part One — What This Document Is

Part Two — Current Build

Part Three — Architecture As Built

Part Four — Architecture Decisions Needed

Part Five — Open Production Questions

Part Six — Next Steps

Part One — What This Document Is

This is the Technical Design Document (TDD) for Three Colors of Madness. It is the companion to the Design Bible, not a replacement for any part of it.

The division between the two documents is simple. The bible owns the what and the why — thesis, Design Laws, narrative, the meaning of every mechanic. This document owns the how — engine architecture, data schemas, production sequencing, and the specific open questions a playtest still needs to settle before a mechanism the bible describes gets its final shape. Where a passage in the bible states a mechanism rather than just a rule, that is deliberate: the mechanism itself is what's enforcing a Design Law, and this document does not relitigate it. Where the two documents ever appear to disagree, the bible is correct and this document is wrong until revised — never the reverse.

This document is expected to change often and change fast. Unlike the bible, it has no standing rule against being provisional — an architecture decision made here can be wrong, discovered to be wrong, and replaced without that being a failure of the document. Its job is to stay honest about the current state of the build, not to be permanently settled.

As of this v1, the project has one real vertical slice to ground this document in: a Godot 4 prototype of Chapter One's opening, built by an AI coding agent (Codex) against Design Bible

v10. Everything in Parts Two and Three below is a description of what that build actually does, verified by reading its source directly — not a plan for what it should someday do.

Part Two — Current Build

Engine and Target

Godot 4.7, using the GL Compatibility renderer (not Forward+), targeting broad hardware reach over cutting-edge rendering fidelity. Window target 1440×900, canvas-items stretch mode. No import dependencies beyond the engine itself — the project runs from a portable Godot install with no additional libraries or network connection required.

Scope of the Current Slice

The build covers the opening movement of Chapter One ("No Exit Wound"): the Ophion estate grounds (rose garden crime scene, birch grove, terrace, gatehouse), a walk to Pickman Street, the precinct, the boardinghouse, and Walter's own room. It is a third-person 3D prototype with primitive placeholder geometry — simple shapes standing in for figures and environment, no imported art assets. Roughly an 8–12 minute exploratory pacing target for the opening alone, explicitly a target to measure rather than a fixed runtime.

Implemented: perspective movement and camera collision, eight optional observations at the crime scene (wounds, the second scene, the knife, the sixth man's watch, the gas/blast question, the seating register, the boy's and woman's belongings, the coroner's assistant's testimony), branching conversation, a sparse paperdoll/inventory, displayed Strength and Perception, a swappable coat with a conditional gardener statement tied to it, a finite flask affecting presentation only, two report choices (routine versus full-inquest), and a persistent case record that survives save/load.

Not yet implemented: the tunnel, combat, the forced-spill mechanic, the glass-shattering ontological break, the fatal-comprehension ending, Chapters Two and Three, gamepad support, full key rebinding, a full encumbrance/leveling system, and Ekon's later consumption of Walter's record. The Part Seven difficulty screen is deferred. Design Bible v10 is the version this slice was built against — it predates the Player Orientation sections, Laws 13–14, and the production notes now split into this document, so it should not be read as validating or invalidating anything added since.

Controls and Accessibility

WASD/arrow movement, mouse look, Q/R camera orbit, scroll-wheel camera distance, Shift for brisk walking, E/F to examine or speak, Tab/I for personal effects, J for the case file, Escape to pause, F11 fullscreen. Menus support mouse or keyboard (Tab, Shift+Tab, Enter/Space).

An accessibility menu is already present and reachable before play: static grain (on by default), distortion intensity, grain intensity, contrast, text size up to 130%, mouse sensitivity and inversion, and optional interaction markers. Text is rendered above all film effects at all times. This is ahead of where Design Law 9 requires the project to be at this stage — the settings exist in the current build, not just in planning.

Saves

Godot's user data directory (`.runtime-data`, kept inside the project folder for this portable build) holds save state. Saves occur after interactions, every 20 seconds of exploration, on pause, and on exit. A save records position, camera state, evidence, statements, clothing, report copies, flask supply, and presentation-relief settings. QA uses a separate save slot from normal play. Earlier save versions migrate automatically on load (see `case_state.gd`'s `VERSION` field and `restore()` migration path, Part Three below); a save completed through the opening continues correctly into the town sequence.

Verification

The build ships with its own automated checks, not just manual playtesting: `Test town.cmd` and `Test opening.cmd` drive scripted traversal through the full opening and town sequence, checking minimal-route progression without ever opening the case file, report immutability and dated-supplement history, save migration, and cross-location save integrity. `tests/smoke_test.gd` and `tests/town_flow.gd` are the underlying GDScript test files. This means several of the bible's fairness guarantees — Design Law 13's minimal-engagement floor in particular — are already validated in code, not only asserted in prose.

Live-renderer QA captures exist under `docs/qa/` (title, case file, dialogue, settings, effects, large-text, gate) via developer-only launch arguments not exposed in-game.

Part Three — Architecture As Built

This section describes what the current codebase actually does, file by file, as read directly from the project.

Autoloads

Two autoloads are registered in `project.godot: GameState` (`scripts/game_state.gd`) and `LegacyInputs` (`legacy_inputs.gd`). Both exist for backward compatibility with an earlier, now-deprecated 2.5D prototype (reachable through `Launch legacy.cmd`, distinct from the current `main.tscn` entry point) and are not part of the active data flow for the current slice. See Part Four for why `GameState` still matters despite being legacy.

Entry Point and Main Loop

`main.tscn` → `main.gd` is the current entry point and, at roughly 1,000 lines, is presently a monolith: it owns player movement and camera, all UI panels (title, case file, journal, report screen, pause, settings), dialogue sequencing, the save/load path, and the integration test harness invoked by the `.cmd` test runners. It preloads `case_state.gd`, `story.gd`, `estate.gd`, `town.gd`, and `town_story.gd` directly rather than through any dependency-injection or service-locator pattern.

Case State

`case_state.gd` (a `RefCounted`, not a `Node`) is the canonical, serializable record of a playthrough. Its fields: `evidence` and `statements` (arrays of discovered-item and given-statement IDs), `visited` (locations), `report` (which report mode was chosen), `copies` (a running list of which physical copies of the report exist — "Walter's notebook," "Precinct intake," and conditionally "County registrar — outgoing copy" / "dispatched copy"), `report_evidence/report_statements` (a snapshot taken at the moment the report is prepared, distinct from `evidence/statements` which keep accumulating after), `flask` (0–3), `coat`, `minutes`, `position/yaw` (camera and player transform), `started/finished/estate_complete/intake_done` (progression flags), `supplement_filed/county_dispatched` plus their own evidence snapshots, `inquiry_topics`, and `supplement_history` (a list of dated supplement filings, each recording its own evidence snapshot and whether it was county-dispatched).

Three methods carry real design weight, not just bookkeeping: `complete_report()` snapshots evidence and statements into `report_evidence/report_statements` and locks in which physical copies exist, refusing to run again once `intake_done` is true; `receive_report()` is what actually makes a report immutable at the precinct and, for a full-inquest report, converts the outgoing county copy into a dispatched one; `file_supplement()` appends a dated, separately evidenced supplement without touching the original report. This is a direct, working implementation of the bible's "reports become immutable when received; later findings file as dated supplements" rule (Part Three, Player Orientation) and of the triplicate/county-registrar resistant-path described there.

`perception()` is currently a flat function of evidence count ($2 + \min(4, \text{evidence.size()} / 2)$) with no linking mechanic. `restore()` handles save-version migration explicitly (a version-1 save missing `estate_complete` is upgraded on load rather than rejected), and validates array types and position shape before trusting a save file — a reasonable defensive pattern worth keeping as the schema grows.

Dialogue and Facts

`story.gd` and `town_story.gd` are static data tables (`const FACTS`, `const INTRO`s, `const SCENES`) — dialogue and evidence content as structured literals, not a general-purpose dialogue engine. `story.gd`'s comment is explicit about intent: "Film effects never modify these canonical strings" — the factual content of a discovery is separated from its presentation, which is the actual code-level implementation of Design Law 4 (unreliable presentation, fair game state). The writing in these tables is already in the bible's register — flat, observational, no character explaining the theme to the player — and reads as consistent with Design Law 5 as currently drafted.

Environment

`estate.gd` and `town.gd` build their environments procedurally at runtime from primitive geometry (`box`, `cylinder`, `sphere`, `mesh_at`, plus purpose-built helpers like `hedge`, `tree`, `lamp`, `person`, `body`) rather than from authored scenes or imported assets. There are no `.tscn` files for the estate or town interiors themselves — the environment is code. This is a legitimate and fast blockout strategy, and it is why the build looks like "simple shapes and nearly stick figures" while still playing correctly. It is not, on its own, a decision about what the shipping game's art pipeline will be — see Part Four.

Presentation Shader

`shaders/film_1923.gdshader` is a `canvas_item` shader applied as a full-screen post-process. It converts the scene to a silver grayscale grade by luminance, but selectively preserves color where red channel dominance is detected (`red_dominance`, blended back in via a `preserve_red` uniform) — a working implementation of the bible's "grayscale except for color that isn't supposed to be there" device, currently generalized to red-dominant content rather than scoped specifically to Observer jewelry/reflections (worth a deliberate look — see Part Four). It applies animated horizontal UV shiver and grain keyed to an `instability` uniform, a vignette, chromatic fringing that scales with `instability` and distance from center, and a combined iris-mask/vertical-frame mask (`aperture_radius`, `frame_width`) that `main.gd` animates over time — this is the Caligari iris device from Part Three's Mechanics

section, actually built and functioning, not just described. A `tunnel_darkness` uniform exists and is wired but unused in the current slice, since the tunnel itself isn't built yet.

Settings

`main.gd` holds a flat settings dictionary (`grain`, `distortion`, `contrast`, `text_scale`, `sensitivity`, `camera_speed`, `reduced_flicker`, `hints`, `invert_y`) applied directly to the shader and UI at runtime via `_apply_settings()`. This is simple and working; it will need a real persistence/versioning story once settings need to survive a Godot project upgrade or ship across chapters built by different hands (see Part Four).

Part Four — Architecture Decisions Needed

These are not open production questions in the bible's sense (Part Five below covers those) — they are decisions this codebase is already implicitly making by default, and that default should be confirmed or overridden on purpose before Chapter Two's build starts.

Two coexisting case-tracking systems. The legacy `GameState` autoload (`scripts/game_state.gd`) implements a link-based corkboard mechanic — discrete evidence pairs (`VALID_LINKS`) that combine into named discoveries, with its own `perception` and `instability` tracking — but it belongs to the deprecated 2.5D prototype, not the active build. The current, actively developed system (`case_state.gd`, driving `main.tscn`) has no linking mechanic at all; its `perception()` is a flat function of evidence count. The bible's corkboard passage (Part Three, Mechanics) describes the board as "the diegetic display of accumulated Perception, not a manual puzzle a player must correctly solve to progress" — that description is compatible with either implementation, but the two current implementations produce meaningfully different player experiences, and only one of them is being built forward. This should be decided explicitly: migrate the link mechanic into `case_state.gd`, rebuild it fresh against the active system, or deliberately keep the simpler evidence-count formula and update the bible's corkboard language to match a system with no linking at all. Whichever way this goes also directly affects the Minimal-Engagement Staging question in Part Five.

`main.gd` needs decomposition before Chapter Two. A single ~1,000-line file currently owns player control, camera, every UI surface, dialogue sequencing, save/load, and test harness integration. That was a reasonable way to move fast on one chapter's vertical slice. It will not scale cleanly into "three distinct interaction sub-systems sharing a grammar, not shared engineering" (Design Bible, Part Six, Common Interaction Grammar) — Chapter Three in particular needs to reuse pieces of Chapter One's investigation grammar and Chapter Two's hazard-navigation grammar inside its own systems, and that reuse is much harder to do cleanly

out of a monolith. Recommend splitting UI (a panel/menu manager), dialogue (a sequencer operating on data tables like `story.gd`), save/load (a dedicated save manager built on the `case_state.gd` pattern), and the camera rig into their own reusable nodes or autoloads before Chapter Two work begins in earnest.

Procedural geometry: blackout strategy or production pipeline? `estate.gd/town.gd` generate all current geometry from code at runtime. This should be named explicitly as either a temporary placeholder to be replaced by authored/imported art, or an intentional low-poly production pipeline the project commits to going forward (which has real implications for how the roughly 200 bespoke Chapter Two artifacts and any Chapter Three assets get built). Leaving this undecided by default risks a costly rebuild later.

Shader color-preservation scope. `film_1923.gdshader`'s red-dominance color preservation currently applies to any sufficiently red-dominant pixel in frame, not specifically to Observer jewelry and reflections as the bible's "Observers" passage (Part Two) describes. Worth confirming whether the intended effect is genuinely global (any warm-toned object reads as a color tell) or should be scoped to specific flagged objects/materials — the current shader can't yet distinguish "an Observer's ring" from "a red door."

Settings persistence. The settings dictionary in `main.gd` is runtime-only within the current architecture; it will need an explicit save/versioning story once accessibility settings must survive across a project upgrade or be shared consistently across chapters built independently.

Part Five — Open Production Questions

These are migrated from the Design Bible, where earlier drafts of this document stated them inline as "Production notes." Each remains an open question pending playtest, not a settled design decision — the bible states the constraint each answer must satisfy; this document owns tracking the answer.

Chapter One — Minimal-Engagement Staging (Design Bible, Part Three, Mechanics).

Design Law 13 guarantees a player who skips every optional beat still reaches Chapter One's fatal understanding, but the bible deliberately does not specify how that understanding is staged for a player carrying none of the optional evidence a thorough playthrough would have handed him. The spine "going whole" cannot mean handing that player an articulate theological summary he has no basis to have assembled — that reads as an unearned script injection, not his own realization, and breaks the show-don't-tell posture Design Law 5 holds everywhere else. Proposed direction, not yet confirmed: stage the minimal-engagement version as blunt sensory collision rather than exposition — Walter recognizing his mother's shape on the thing in front of him without the theological context a diligent player has already pieced together. Both players would reach the same fatal end on the same schedule; only the texture — intellectual horror against raw, unframed terror — differs. This is entangled with the corkboard/link-mechanic

decision in Part Four, since what a "diligent player has already pieced together" depends on which perception system ships.

Chapter Two — Procedural Trinket Generation (Design Bible, Part Four, Mechanics). The fiction needs thousands of trinkets; production does not. Proposed direction: a small library of trinket classes (scrimshaw, ship's hardware, personal effects, ballast-stone curios) recombined with randomized wear, placement, and Codex flavor text, plus batch-authored cataloging rather than bespoke per-object descriptions, so felt abundance is a systemic effect rather than an asset-count commitment. Only the roughly 200 artifacts of real value need individual, bespoke authorship. Not yet prototyped.

Chapter Two — Acquisition/Documentation Mechanical Distinction (Design Bible, Part Four, Player Orientation). The acquisition/documentation split is legible on the page but has no mechanical texture yet — as specified, looting and documenting would collapse into the same button press wearing two different flavor texts. Proposed direction: looting stays fast but adds carried weight (its existing cost); documenting adds no weight but costs time and exposure instead — Ward holds an open, stationary stance long enough to log a find properly, lantern lit and position given away, trading the hauler's physical burden for the documentarian's own real risk. Not yet prototyped; no analogous system exists in the current Chapter One slice to reference.

Chapter Three — Match Consumption Model (Design Bible, Part Five, Mechanics). Whether a match burns in real time once struck, or is only spent at discrete, player-initiated moments (inspecting, navigating a stretch that demands light, lighting the fuse), is unresolved. This is flagged in the bible as the chapter's single riskiest mechanical bet — whether dread carried by writing, pacing, and physical scarcity alone reads as strongly as the visible-stat-driven dread of Chapters One and Two — and should not be "fixed" by adding a fuel stat without playtest data showing the tension is actually reading thin. No Chapter Three slice exists yet to prototype against.

Cross-Chapter — Accessible-Mode Degradation Vocabulary (Design Bible, Part Six, Accessibility Suite). Flicker reduction and distortion intensity set to their floor cannot simply mean the medium stops degrading — Design Law 8 still has to read as something breaking down, just through a different vocabulary, or the accessible version of the game quietly stops being the same game. Proposed direction: an accessible-mode substitution built from composition and geometry rather than optical assault — shadows stretching wrong, negative space expanding, background geometry flattening toward silhouette, sound design leaning into bass rumble and dry silence rather than static and screeching foley. Untested; the current shader's *instability*-driven grain/fringe/shiver system has no fallback vocabulary implemented yet for when those uniforms are floored.

Meta Layer — Difficulty-Menu Glitch Cue (Design Bible, Part Seven, Difficulty Selection). The glitch produced by repeatedly selecting a greyed-out difficulty option must read as the interface acting with intent, never as broken software or a crashed build. Proposed direction: a flicker using the same aspect/grain/audio vocabulary the rest of the game already uses — a

static pop, a half-frame of distortion, a flash of the same handwriting font spelling something illegible — rather than a freeze or blank stall. Not built; no difficulty screen exists in the current slice (explicitly deferred per the README).

Part Six — Next Steps

In rough priority order: resolve the corkboard/link-mechanic decision (Part Four) before building anything further on top of `case_state.gd`'s perception system, since it determines both how Chapter One's board actually works and how the Minimal-Engagement Staging question (Part Five) gets answered. Decide the procedural-art-pipeline question before committing more chapters to code-generated geometry. Begin decomposing `main.gd` before Chapter Two's build starts, so the second chapter isn't built as a second monolith. Once a Chapter Two slice exists, this document gets a matching Architecture As Built entry for it, the same way Chapter One's does now.